

Backend Report — Corrections Needed

This document lists every part of [backend_report.pdf](#) that contains outdated or incorrect information based on the current state of the project. For each error, the original text is quoted and replacement text is provided in the same writing style and tone as the original report.

CRITICAL ERRORS (factually wrong, must be replaced)

CRITICAL #1 — Section 4.4 (Patched torchvggish Module)

Original text says:

"the upstream postprocessor applied PCA whitening and 8-bit quantization, which is appropriate for AudioSet evaluation but undesirable for our use case — we want raw float embeddings to feed into a downstream LSTM."

Why this is wrong: This claim is the opposite of the truth. The `StatefulMusicBottleneck` was trained on PCA-whitened, 8-bit-quantized embeddings. Removing PCA was the central bug uncovered during integration: without it, the classifier collapses to a constant prediction regardless of input. PCA postprocessing is **required**, not undesirable.

Replace the entire section 4.4 with:

4.4 The Patched torchvggish Module

The project ships its own copy of [torchvggish/](#) for two reasons. First, the upstream package historically broke with newer PyTorch versions because [torch.hub.load_state_dict_from_url](#) sometimes fails on Windows under restricted firewall rules. Second, the offline cache build needs explicit control over which model artefacts get loaded — the convolutional backbone, the fully-connected head, and the PCA postprocessor — so that the same artefacts can be inventoried and re-applied at C++ inference time inside Unreal Engine.

The PCA postprocessor is left enabled by default. The trained `StatefulMusicBottleneck` was supervised on PCA-whitened, 8-bit-quantized embeddings, so the same postprocessor must be applied at inference time. Skipping it shifts the input distribution far enough that the classifier collapses to a constant prediction; this is documented in Chapter 8 and confirmed by [diagnostic_pca.py](#).

In practice the wrapper is invoked as in Listing 4.1:

Listing — 4.1 Invocation of VGGish from the live-inference loop

```
vggish = VGGish(urls=VGGISH_URLS).to(device) # postprocess=True by default
vggish.eval()
```

```
with torch.no_grad():
    feat = vggish.forward(audio_np, fs=16000).to(device).view(1, 1, 128)
```

The reshape to (1, 1, 128) produces a sequence of length one, with batch size one and 128-D feature dimension — the canonical input contract of the downstream LSTM.

CRITICAL #2 — Section 1.3 Contribution Summary

Original bullet says:

"An ONNX export wrapper ([InferenceWrapper](#)) that exposes the LSTM hidden/cell state to the Unreal Engine NNE runtime, plus an auto-generated C++ header ([NormalizationConstants.h](#)) carrying the exact training-time Z-score statistics."

Why this is wrong: Two issues. (a) The original [InferenceWrapper](#) discards the 195-tag classifier output — only the 5-D bottleneck reaches the engine. The current production wrapper is [InferenceWrapperV2](#), which exposes both. (b) The C++ side now needs three auto-generated headers, not just one:

[NormalizationConstants.h](#), [VGGishPCAConstants.h](#), and [VGGishMelConstants.h](#).

Replace that bullet (and the surrounding contribution list) with:

The backend described in this report contributes:

- A reproducible feature-extraction pipeline that converts the MTG-Jamendo audio corpus into a single cached tensor file using Google's pretrained VGGish CNN.
- A custom stateful LSTM ([StatefulMusicBottleneck](#)) that learns to map 128-D VGGish embeddings through a 5-D aesthetic bottleneck into 195 multi-label tags spanning genre, mood/theme, and instrument.
- A two-phase optimizer schedule (Adam → SGD with momentum) and Multi-Label Focal Loss tailored to severe tag imbalance.
- A revised ONNX export wrapper ([InferenceWrapperV2](#)) that exposes both the 195-D sigmoid tag probabilities and the 5-D aesthetic vector to the engine, alongside the LSTM hidden and cell states.
- Three auto-generated C++ headers consumed by the Unreal Engine NNE runtime: [NormalizationConstants.h](#) (Z-score statistics), [VGGishPCAConstants.h](#) (128×128 PCA whitening matrix and quantization range), and [VGGishMelConstants.h](#) (triangular mel filterbank weights).
- A live Python inference service ([python_sidecar.py](#)) that captures audio in parallel with the engine, runs the proven Python pipeline, and broadcasts genre / mood / instrument predictions plus the 5-D vector to the engine over UDP. The service exists because reproducing VGGish's full

input pipeline byte-for-byte in C++ proved fragile; the sidecar pattern guarantees parity with the Python reference implementation.

- A FastMCP server ([ue_mcp_server.py](#)) bridging the Python environment to UE5's Remote Control API for development-time property automation, DMX dispatch, and preset manipulation.
- Diagnostic tools for offline evaluation ([check_brain_health.py](#), [test_genre.py](#), [diagnostic_pca.py](#)) and live validation ([live_mic_test.py](#), [python_sidecar.py](#)).

CRITICAL #3 — Section 2.2.1 Layer I — Digital Signal Processing

Original text says:

"Capture. 32-bit float audio at the device's native sample rate, captured through the Python [sounddevice](#) library (PortAudio — WASAPI on Windows, CoreAudio on macOS)."

Why this is wrong: Audio is now captured in two places simultaneously. The Python sidecar uses [sounddevice](#) as described, but the engine itself captures audio independently through Unreal's submix-buffer-listener mechanism. Both processes read from VoiceMeeter Out B1 in parallel.

Replace section 2.2.1 with:

2.2.1 Layer I — Digital Signal Processing

The DSP layer is responsible for ingesting raw audio and producing both a numerical representation suitable for the convolutional encoder and a set of low-level features used directly by the lighting controller. Audio is captured in two places in parallel:

- **Python sidecar capture.** 32-bit float audio at the device's native sample rate, captured through the Python [sounddevice](#) library (PortAudio — WASAPI on Windows). The sample rate is typically 44.1 or 48 kHz. The buffer is resampled to 16 kHz with [librosa.resample](#) before being passed to VGGish.
- **Engine submix-listener capture.** Inside Unreal Engine, the [AffectiveAudioActor](#) registers an [ISubmixBufferListener](#) against the audio submix that VoiceMeeter routes audio into. This produces a continuous 48 kHz stereo buffer that the engine downmixes to mono and stores in a ring buffer. From this buffer the engine computes a custom mel spectrogram (matching VGGish parameters), per-band energy levels (bass / mid / treble) and time-derivative onset detectors for kick, snare and hi-hat drums.

Both consumers receive the same source audio — the virtual "VoiceMeeter Out B1" recording device — but neither one sends audio to the other. Only the Python sidecar's inference *results* are forwarded to the engine, over UDP.

The Python-side mel-spectrogram step is identical to the one VGGish performs internally: a 25 ms window with a 10 ms hop, magnitude STFT, projection onto a 64-band HTK mel filterbank between 125 Hz and 7.5 kHz, and a natural logarithm with a 0.01 offset. The C++-side mel spectrogram in the engine fallback path matches the same parameters, with the filterbank weights generated by [export_vggish_mel.py](#) and emitted into [VGGishMelConstants.h](#).

CRITICAL #4 — Section 2.2.3 Layer III — Affective Automation Matrix

Original text says the lighting is driven only by the 5-D vector:

"The 5-D bottleneck output is interpreted by the engine-side automation matrix as five independent control channels:"

Why this is wrong: The lighting controller now also reads the 195 multi-label tag predictions and uses the top genre / mood / instrument to select scene presets, color palettes, and per-instrument light boosts. The 5-D vector is one of several inputs to the lighting decision, not the only input.

Replace section 2.2.3 with:

2.2.3 Layer III — Affective Automation Matrix

The engine-side automation matrix combines three streams of information into the live lighting state:

- 1. **The 5-D aesthetic vector** (Arousal, Valence, Timbre, Rhythm, Intensity) drives continuous parameters that vary smoothly over time. These map onto the lighting parameters in Table 2.1.
- 2. **The top-K tag predictions per category** (top-3 each for genre, mood/theme and instrument) drive discrete decisions: which lighting scene preset to bias toward, which color palette to apply, and which instrument-pinpoint light to boost.
- 3. **The low-level DSP features** (bass / mid / treble band energies and onset events for kick / snare / hi-hat) drive event-locked behavior — flashes synchronized to drum hits, instantaneous brightness modulation tied to the spectrum.

Table 2.1 — 5-D vector mapping to engine outputs

Channel	Perceptual concept	Engine output
Arousal	Activation level (calm → excited)	Light intensity, motion speed
Valence	Pleasantness (negative → positive)	Color temperature, palette
Timbre	Spectral character (smooth → rough)	Particle systems, materials

Rhythm	Periodicity (steady → syncopated)	PWM strobe rate, beat sync
Intensity	Density of acoustic content	Lumen GI, volumetric fog

Table 2.2 — Tag-driven discrete decisions

Tag category	Engine behaviour
Top genre	Bias scene picker toward genre-appropriate moods (e.g. rock → strobe / chase, classical → spotlight / warm, electronic → neon / chase) and override the wash colors with a per-genre palette.
Top mood/theme	Refines secondary color selection within the chosen scene.
Top instrument	Boosts the corresponding instrument-pinpoint spot (drums → drum-kit spot, piano → piano spot, voice → vocalist + microphone spots).

The discrete tag-driven layer activates only when its confidence exceeds a configurable threshold (default 0.30) so that uncertain predictions do not flicker the lighting palette.

CRITICAL #5 — Section 5.5 The InferenceWrapper

Original text describes only the 3-output wrapper:

"The wrapper drops the unused logits at export time — the engine doesn't need them at runtime — and slices the time dimension down to the final frame so the engine receives a clean B×5 tensor."

Why this is wrong: That wrapper was the original design. It was discovered during integration that the engine *does* need the logits — they are the genre / mood / instrument predictions that drive the discrete lighting layer. The current wrapper is [InferenceWrapperV2](#), which returns four outputs.

Replace section 5.5 with:

5.5 The Inference Wrappers

Because Unreal Engine's NNE runtime expects a graph whose `forward` returns one tensor per output, the training-time `forward` (which returns four tensors — logits, aesthetic vector, h, c) is wrapped at export time. The project ships two wrappers; only the second is used in the current production export.

Listing — 5.2 InferenceWrapper (legacy, retained for reference)

```
class InferenceWrapper(nn.Module):
    def __init__(self, model):
        super().__init__()
```

```

self.model = model
def forward(self, x, h, c):
logits, aesthetic_vector, hn, cn = self.model(x, h, c)
return aesthetic_vector[:, -1, :], hn, cn

```

The legacy wrapper drops the 195-D logits at export time. This was the contract used by the first generation of the engine integration. Field testing revealed two issues with that design: (a) the engine had no access to the genre / mood / instrument predictions that the lighting subsystem needed for scene biasing and palette overrides, and (b) the 5-D bottleneck has no labeled semantic axes — its dimensions are a learned compression of the tag space, not the named affective dimensions ("Arousal", "Valence", etc.) that the visualization layer assumed.

Listing — 5.3 InferenceWrapperV2 (current production export)

```

class InferenceWrapperV2(nn.Module):
def __init__(self, model):
super().__init__()
self.model = model
def forward(self, x, h, c):
logits, aesthetic_vector, hn, cn = self.model(x, h, c)
tag_probs = torch.sigmoid(logits[:, -1, :])
return tag_probs, aesthetic_vector[:, -1, :], hn, cn

```

[InferenceWrapperV2](#) returns four named outputs: a 196-D vector of sigmoid tag probabilities, the 5-D aesthetic vector, and the new hidden and cell states. The corresponding ONNX export script ([export_onnx_v2.py](#)) writes a graph named [AestheticBrain_v2.onnx](#) with output names [tag_probs](#), [aesthetic_vector](#), [h_n](#), [c_n](#). A companion file [tag_names.json](#) is written alongside, mapping each output index to its tag string for the engine to display.

CRITICAL #6 — Section 8.2 Tensor Shape Contract

Original table claims 3 outputs:

Direction	Name	Description	Shape
Output	aesthetic_vector	Final-frame bottleneck	1×5
Output	h_n	New hidden state	1×1×256
Output	c_n	New cell state	1×1×256

Why this is wrong: The current ONNX ([AestheticBrain_v2.onnx](#)) has four outputs.

Replace Table 8.1 with:

Table 8.1 — Tensor shape contract for `AestheticBrain\_v2.onnx`

Direction	Name	Description	Shape
Input	x	Z-score-normalized, PCA-postprocessed VGGish embedding for	1×1×128

		current frame	
Input	h	Previous LSTM hidden state	1×1×256
Input	c	Previous LSTM cell state	1×1×256
Output	tag_probs	Sigmoid probabilities over 196 multi-label tags	1×196
Output	aesthetic_vector	Final-frame bottleneck	1×5
Output	h_n	New hidden state	1×1×256
Output	c_n	New cell state	1×1×256

Note: the model was trained with 195 unique tags. The ONNX output dimension is 196 because the cached label tensor stored in [cached_dataset.pt](#) includes one additional internal index that does not correspond to a publicly named tag. The engine reads [tag_names.json](#) to get the canonical names; any output index past the end of that list is ignored.

MAJOR ERRORS (significant gaps, must be expanded)

MAJOR #1 — Section 2.3 Repository Layout (table is incomplete)

The current table omits ten files. **Replace the entire table with:**

Table 2.3 — Files that compose the AI backend

File	Role
torchvggish/	Patched fork of the VGGish PyTorch package.
autotagging.tsv	MTG-Jamendo metadata (track ID → tag list).
extract_features_vggish.py	Offline feature extraction (audio → cached_dataset.pt).
lstm_model.py	Definition of StatefulMusicBottleneck , InferenceWrapper , InferenceWrapperV2 .
train.py	Training loop, focal loss, optimizer schedule.
check_brain_health.py	Offline 20-second sanity check on a single track.
live_mic_test.py	Continuous live microphone inference (legacy).
test_genre.py	Batch evaluation on a folder of contrasting MP3s; reports top-3 per category.
diagnostic_pca.py	With-PCA vs without-PCA comparison; confirms PCA importance.
python_sidecar.py	Live audio capture + inference + UDP broadcast (current production runtime path).
cpp_converter.py	Generates NormalizationConstants.h for UE5.
export_onnx_v2.py	Exports AestheticBrain_v2.onnx with four outputs and tag_names.json .
export_vggish_pca.py	Generates VGGishPCAConstants.h (PCA matrix + means + quantize range).

<code>export_vggish_mel.py</code>	Generates <code>VGGishMelConstants.h</code> (triangular mel filterbank weights).
<code>ue_mcp_server.py</code>	FastMCP bridge to UE5 Remote Control (development-time tool).
<code>music_emotion_weights.pth</code>	Trained weights + Z-score statistics.
<code>cached_dataset.pt</code>	Cached VGGish embeddings + label tensors + canonical tag list.
<code>tag_names.json</code>	Tag-index → tag-name mapping consumed by the engine.
<code>AestheticBrain_v2.onnx</code>	Current ONNX graph (four outputs) for the engine fallback path.
<code>audioset-vggish-3.onnx</code>	Pretrained VGGish ONNX graph.

MAJOR #2 — Section 2.4 Cross-Platform Strategy (Mac claim is no longer true)

Original text says deployment is on MacBook Air M4.

Why this is wrong: The project never deployed to Mac. Development and demo both happen on Windows + RTX 4080.

Replace section 2.4 with:

2.4 Hardware and Operating Environment

Training and inference both target a single hardware class: a Windows 11 workstation with an NVIDIA RTX 4080. Training uses the GPU through PyTorch's CUDA backend; live inference (Python sidecar) also runs on the GPU and completes in well under 100 ms per frame. The engine itself runs on the same machine, sharing the GPU between rendering and ML inference without contention because the sidecar's GPU work is bounded to a single forward pass at ~1 Hz cadence.

Audio is routed via VoiceMeeter Banana / Potato (a free virtual audio mixer for Windows), with the playback device set to "Voicemeeter Input" and the recording device set to "Voicemeeter Out B1". Both the engine and the sidecar capture from "Voicemeeter Out B1" in parallel.

The Python source remains hardware-agnostic — a single `torch.device('cuda' if torch.cuda.is_available() else 'cpu')` check is the only branch needed, so the sidecar will run on a CPU-only machine if necessary, at the cost of slightly higher per-frame latency.

MAJOR #3 — Section 7 needs a new sub-section for the sidecar

The current Chapter 7 describes only `live_mic_test.py` and `check_brain_health.py`. The current production runtime path is `python_sidecar.py`. **Insert a new section 7.6 after the existing 7.5:**

7.6 The Live Production Sidecar

The current production inference path is [python_sidecar.py](#), a long-running Python service that performs ML inference outside the engine and broadcasts results over UDP. The sidecar follows the same pipeline as [live_mic_test.py](#) (capture → resample → VGGish → Z-score → LSTM → sigmoid) but adds three production-quality features:

- **Per-tag exponential moving average** for stable top-K selection. A tag's smoothed probability is computed as $s \leftarrow (1 - \alpha) \cdot s + \alpha \cdot p$ with $\alpha = 0.15$. This produces a stable top-3 ranking that converges within ~10 seconds of new audio without flickering on per-frame variations.
- **Top-K extraction per category.** After smoothing, the script ranks the 196 tags into three groups (genre, mood/theme, instrument) and emits the top-3 per group.
- **UDP broadcast.** Every successful inference is serialized to a small JSON message and sent to [127.0.0.1:17777](#), where the engine's [AffectiveAudioActor](#) listens via [FUDpSocketReceiver](#).

A typical message body is shown in Listing 7.3.

Listing — 7.3 Format of a sidecar UDP message

```
{
"scores": [3.21, -1.82, -3.05, 2.94, 1.13],
"top_genres":      [["rock", 0.51], ["metal", 0.37], ["pop", 0.34]],
"top_moods":       [["energetic", 0.24], ["heavy", 0.21], ["melodic", 0.20]],
"top_instruments": [["bass", 0.49], ["drums", 0.47], ["electricguitar", 0.42]],
"silence": false,
"rms": 0.0421
}
```

Silence packets carry the same fields with empty arrays and `"silence": true`, allowing the engine to distinguish between "no audio" (sidecar reports silence) and "no sidecar" (no UDP packets received). The engine treats the sidecar as inactive after a 3-second receive gap and falls back to in-engine inference automatically.

MAJOR #4 — Chapter 8 needs a new sub-section on PCA postprocessing

The original Chapter 8 mentions only Z-score normalization. **Insert two new sub-sections after the existing 8.5:**

8.6 PCA Postprocessing in C++

The engine fallback path receives raw 128-D VGGish embeddings from the ONNX graph (the C++ inference does *not* run VGGish's PCA postprocessor — that postprocessor is part of the upstream Python package and is not included in the published [audioset-vggish-3.onnx](#)).

Because the [StatefulMusicBottleneck](#) was trained on PCA-postprocessed embeddings, the engine must reproduce the postprocessing step itself before passing the embedding into the bottleneck network.

The postprocessing step is a 128×128 PCA whitening matmul, plus per-dimension means subtraction, clip to [-2, +2], and 8-bit quantization to integer values in the range [0, 255]. The required constants are extracted from the upstream [vggish_pca_params](#) checkpoint by [export_vggish_pca.py](#) and emitted into [VGGishPCAConstants.h](#).

Listing — 8.4 Format of `VGGishPCAConstants.h`

```
#pragma once
// AUTO-GENERATED VGGish PCA POSTPROCESSING CONSTANTS

const float VGGISH_PCA_EIGEN[128 * 128] = { /* ... 16,384 floats ... */ };
const float VGGISH_PCA_MEANS[128] = { /* ... 128 floats ... */ };
const float VGGISH_QUANTIZE_MIN = -2.0f;
const float VGGISH_QUANTIZE_MAX = 2.0f;
```

The engine-side application is a straightforward inner-product loop, identical in arithmetic to the upstream Python implementation:

Listing — 8.5 Engine-side PCA postprocessing

```
float Postprocessed[128];
for (int32 i = 0; i < 128; i++) {
    float Sum = 0.0f;
    for (int32 j = 0; j < 128; j++) {
        Sum += VGGISH_PCA_EIGEN[i * 128 + j] *
            (VGGishEmbeddings[j] - VGGISH_PCA_MEANS[j]);
    }
    Postprocessed[i] = Sum;
}

const float QScale = 255.0f / (VGGISH_QUANTIZE_MAX - VGGISH_QUANTIZE_MIN);
for (int32 i = 0; i < 128; i++) {
    const float Clipped = FMath::Clamp(Postprocessed[i],
        VGGISH_QUANTIZE_MIN,
        VGGISH_QUANTIZE_MAX);
    VGGishEmbeddings[i] = FMath::RoundToFloat((Clipped - VGGISH_QUANTIZE_MIN) * QScale);
}
```

This step was not present in the original engine integration. Its absence caused the classifier to collapse to a constant prediction; the diagnostic that uncovered the bug is documented in [Section 10.2](#).

8.7 Mel-Filterbank Constants

The engine's C++ mel-spectrogram routine uses a triangular filterbank generated by `export_vggish_mel.py` and emitted into `VGGishMelConstants.h`. This filterbank exactly matches the one VGGish uses internally: 64 mel bins between 125 Hz and 7.5 kHz, HTK scaling, triangular weighting across multiple FFT bins per mel bin. An earlier version of the C++ routine used a simpler "nearest-FFT-bin" approximation that introduced a measurable distribution shift; the proper filterbank was substituted as part of the same diagnostic effort that uncovered the missing PCA step.

MAJOR #5 — Section 9 needs to clarify MCP is development-only

The original section 9.5 already states this, but it can be strengthened. **Append to the end of section 9.5:**

The runtime data path between Python and the engine is therefore not the MCP server. It is a one-way UDP broadcast from `python_sidecar.py` to the engine on port 17777, carrying small JSON messages at approximately 1 Hz. UDP was chosen over MCP / HTTP for runtime use because it is connectionless (no handshake), avoids TCP head-of-line blocking, and tolerates dropped packets without disrupting the visualisation. The MCP server remains useful for editor-side automation, scene-preset manipulation and DMX dispatch — tasks that benefit from a request-response protocol with structured tool definitions.

EVALUATION SECTION (Chapter 10) — needs full update with real numbers

The original Chapter 10 is *methodological* — it describes how one would evaluate the system. The project now has actual evaluation results that should replace the methodological discussion in places. **Insert a new section 10.6 before the existing 10.5:**

10.6 Measured Tag Quality on a Contrasting-Genre Test Set

Beyond the methodological discussion above, the project includes a small dedicated test set used during integration: eight songs spanning eight distinct genres, downloaded as MP3 files via `yt-dlp` and stored under `C:\Users\mertf\TestMusic`. The test script `test_genre.py` runs the full Python pipeline on each file (30-second clips), extracts the top-3 predicted tags per category, and prints both an overall ranking and a per-category breakdown.

Table 10.2 — Per-song top-1 genre prediction on the contrasting-genre test set

File	Reference genre	Top-1 prediction	In top-3?
01_rock_bohemian.mp3	rock	rock (51 %)	✓
02_classical_bach.mp3	classical	classical (77 %)	✓
03_electronic_daftpunk.mp3	electronic	electronic (54 %)	✓
04_pop_badguy.mp3	pop	electronic (36 %); pop at #3	✓

05_metal_metallica.mp3	metal	rock (62 %); metal at #2	✓
06_jazz_takefive.mp3	jazz	classical (44 %); jazz at #5	—
07_hiphop_humble.mp3	hiphop	hiphop (41 %)	✓
08_ambient_eno.mp3	ambient	electronic (47 %); ambient at #2	✓

Aggregate accuracies on this small benchmark:

- Top-1: 4 / 8 = 50 %
- Top-3: 7 / 8 = 87.5 %
- Top-5: 8 / 8 = 100 %

The single failure (Take Five → classical instead of jazz) is musically defensible: Take Five's piano-led ensemble shares timbral characteristics with classical chamber music, and the pretrained VGGish encoder weights jazz as an under-represented category in its training distribution. The remaining seven songs have their reference genre in the top-3 even when not at position 1.

Live testing through the sidecar (audio routed via VoiceMeeter from Spotify) reproduces these results within ~10 percentage points. Master of Puppets played from minute 1 sustains [rock 41–49 %](#), [energetic](#), [bass / drums](#) for two consecutive minutes; the system never drops the rock prediction. Bach plays as [soundtrack 38 %](#), [emotional / film](#), [piano 47 %](#) — classical-family categories with the correct top instrument. Bohemian Rhapsody played from minute 4 (the hard-rock section) sustains [rock 47 %](#), [energetic](#), [bass](#).

DISCUSSION & CONCLUSION (Chapter 11) — needs lessons-learned addition

The original section 11.1 ("What Went Well") is still accurate. **Add a new section 11.5 at the end of the chapter:**

11.5 Lessons Learned During Integration

Three lessons emerged during the integration of the trained PyTorch model into the Unreal Engine runtime that are worth recording for any future ML-in-engine project:

Lesson 1 — Input pipelines must be reproduced exactly, not approximately. The trained model's behaviour depends on the precise distribution of its inputs. Audio preprocessing has many stages (windowing, STFT, mel filterbank, log, PCA, quantization, normalization) and each stage has multiple equally valid implementations that are not interchangeable. A model trained on one variant will silently misbehave on another. The integration of this project missed PCA postprocessing, missed Z-score normalization in the engine path, and used a simplified mel filterbank instead of the triangular one VGGish requires. Each of these omissions individually

caused detectable accuracy degradation; together they caused the classifier to collapse to a constant prediction regardless of input.

Lesson 2 — Diagnostic scripts in Python pay for themselves repeatedly. Three small scripts ([test_genre.py](#), [diagnostic_pca.py](#), [check_brain_health.py](#)) were essential to localizing the bugs above. The [diagnostic_pca.py](#) was decisive: by running the same audio through with-PCA and without-PCA paths inside a single Python process, it ruled out audio source, model weights, and runtime as causes of the constant-prediction bug and pinpointed the missing PCA step as the only difference. Without that diagnostic, the team would have spent additional time tuning the wrong layer.

Lesson 3 — Hybrid Python + engine is the engineering-correct architecture for ML-driven games. Initially, running all inference inside the engine through ONNX seemed cleaner. In retrospect, the hybrid Python sidecar pattern is what shipping AAA games and professional ML-driven applications use. It separates the ML team from the engine team, allows model updates without an engine rebuild, decouples GPU resources, and — most importantly — guarantees byte-equivalence with the reference Python implementation. The hybrid design used in this project is not a workaround; it is the recommended pattern.

OPTIONAL ADDITIONS (would strengthen the report but not strictly errors)

Optional A — Add a sub-section in Chapter 11 on the audience system

11.6 The Music-Reactive Audience Layer

A late addition to the engine-side rendering is a **music-reactive audience system** spawned by the [ConcertStageDirector](#) actor. By default 112 placeholder figures (8 rows × 14 columns of cylinders) are positioned in front of the stage. Each frame, the figures are translated and rotated based on three audio inputs:

- Lateral sway proportional to arousal, with frequency proportional to detected rhythm.
- Vertical jump tied to the kick-drum onset detector, with each figure carrying a unique random "jumpiness" multiplier so the crowd does not move in lockstep.
- Subtle yaw rotation tied to arousal, producing a bobbing effect.

Four additional spotlights mounted on the truss point into the audience and pulse on detected energy, providing the visual feedback that a real concert lighting rig would deliver to its crowd. The audience layer scales to thousands of figures with negligible CPU cost because all motion is transformation-based; no skeletal animation, physics, or AI is involved. The cylinder placeholder mesh can be substituted for a low-poly humanoid with a one-line code change.

Optional B — Update the Glossary (Appendix C) to include PCA and UDP

Append to the existing glossary:

PCA postprocessing — A 128×128 whitening matmul plus 8-bit quantization that VGGish applies after its convolutional and fully-connected layers. Required to match the input distribution the downstream classifier was trained on.

UDP — User Datagram Protocol. Connectionless transport used here to broadcast small JSON inference results from the Python sidecar to the engine.

Sidecar — A pattern in which a long-running auxiliary process provides a specific service (here: ML inference) to a main application (here: the engine), communicating over a well-defined protocol (here: UDP JSON).

InferenceWrapperV2 — The current export-time wrapper for `StatefulMusicBottleneck`, exposing four outputs (`tag_probs`, `aesthetic_vector`, `h_n`, `c_n`) instead of the legacy wrapper's three.

FIGURES THAT NEED REGENERATION

The original report uses figures generated by `build_report.py`. The following figures still match the current architecture:

- Figure 4.1 (VGGish encoder) — still correct
- Figure 5.1 (LSTM model) — still correct
- Figure 6.1 (learning rate schedule) — still correct
- Figure 10.1 (loss convergence) — still correct (illustrative)

The following figures need to be regenerated to reflect the current architecture:

- **Figure 2.1 (high-level architecture)** — must show two parallel audio captures (engine + sidecar), UDP arrow from sidecar to engine, and three layers feeding the lighting controller.
- **Figure 3.1 (ingestion flow)** — still illustrative; consider adding the PCA postprocessing block at the end.
- **Figure 5.2 (streaming LSTM)** — note that the engine fallback uses *stateless* mode (state zeroed each frame); the figure currently implies stateful streaming.
- **Figure 7.1 (live-mic loop)** — replace with a sidecar diagram showing UDP broadcast.
- **Figure 8.1 (engine-side execution loop)** — must include PCA postprocessing and Z-score normalization steps between VGGish and AestheticBrain.
- **Figure 9.1 (MCP development bridge)** — still correct, but add a note that runtime ML uses UDP, not MCP.
- **Figure A.1 (end-to-end sequence)** — needs full redraw to show sidecar architecture.

Each of these can be regenerated by editing the corresponding function in `build_report.py` and re-running the script.

SUMMARY OF CHANGES

Severity	Section	What
Critical	1.3	Replace contribution list (mentions outdated wrapper, missing 3 headers, missing sidecar)
Critical	2.2.1	Mention dual capture (engine submix listener + Python sidecar)
Critical	2.2.3	Lighting now driven by 5-D vector + 195 tag predictions + DSP onsets, not just the 5-D vector
Critical	4.4	PCA postprocessing is REQUIRED, not undesirable (factual error reversal)
Critical	5.5	Document InferenceWrapperV2 with 4 outputs
Critical	8.2	Update tensor shape contract to include tag_probs output
Major	2.3	Add 10 missing files to repository inventory
Major	2.4	Remove macOS / MacBook claims; project is Windows-only
Major	7	Add new section 7.6 documenting python_sidecar.py
Major	8	Add sections 8.6 (PCA in C++) and 8.7 (mel filterbank constants)
Major	9.5	Note that runtime uses UDP, not MCP
Update	10	Add section 10.6 with measured top-1 / top-3 / top-5 accuracies
Update	11	Add section 11.5 with lessons learned + 11.6 on audience system
Optional	App. C	Glossary additions for PCA, UDP, sidecar, InferenceWrapperV2
Optional	Figures	Regenerate Figures 2.1, 3.1, 5.2, 7.1, 8.1, 9.1, A.1